

Fiche d'investigation de fonctionnalité

Fonctionnalité : Recherche / Filtrage	Fonctionnalité #1
Problématique : Rechercher et trouver rapidement dans une collection de recettes, une recette correspondant aux critères de choix de l'utilisateur	

Option 1, branche v1 github : boucles natives while et for et fonctions récursives Utilisation des boucles while et for ainsi que des fonctions récursives pour itérer sur les tableaux et réaliser les traitements nécessaires : intersection, soustraction, fusion, ajout d'éléments, retrait etc.	
Avantages ⊕ les fonctions récursives peuvent être plus performantes que les méthodes de l'objet Array ⊕ compatibilité assurée avec toutes les versions de navigateurs ⊕ programmation mobilisant des compétences de base	Inconvénients ⊖ les boucles while et for sont moins performantes que les fonctions de l'objet Array ⊖ Code des fonctions récursives peut apparaître moins lisible et être moins évolutif
Test de performance de l'algorithme : voir la colonne Algorithme 1 de l'annexe 1 (résultats du test jsben.ch) Algorithme : voir annexe 2	

Option 2, branche v2 github : Méthodes de l'objet Array Utilisation des méthodes de l'objet Array pour le traitement des tableaux. Les opérations réalisées sont les mêmes que celles de l'option 1	
Avantages ⊕ globalement, rapidité des fonctions de traitement des tableaux, mais ce résultat doit être analysé dans le détail ⊕ code plus concis ⊕ facilite la maintenance	Inconvénients ⊖ nécessite de maîtriser l'utilisation des méthodes de l'objet Array ⊖ les fonctions récursives sont plus performantes pour la fusion, l'intersection et la soustraction de tableaux ⊖ certaines méthodes : indexOf, filter, every peuvent ne pas être prises en charge par d'anciens navigateurs (cf mdn web docs)
Test de performance de l'algorithme : voir la colonne Algorithme 2 de l'annexe 1 (résultats du test jsben.ch) Algorithme : voir annexe 2	

Solution retenue : à ce stade, la version 2 (méthodes de l'objet Array) de l'algorithme est la solution offrant la meilleure performance globale. C'est la conclusion qui ressort de l'observation de la ligne 1 du tableau de mesures en annexe réalisées sur jsBen.ch concernant les 2 fonctions setSearchedRecipes() et getQuery(). Cette solution possède en outre l'avantage d'un code plus concis et plus maintenable. C'est donc la version retenue dans le cadre de ce projet.

Néanmoins, pour gagner quelques points de performance, une troisième version hybride pourrait être envisagée. Elle utiliserait des fonctions récursives pour la fusion, l'intersection et la soustraction de tableaux. La fonction d'itération simple avec assignation de valeur dans la boucle serait quant à elle réalisée avec la fonction forEach qui est plus performante que la version récursive.

Pour conclure, l'utilisation des boucles while ou for est à écarter car elle dégrade la performance de l'algorithme.



Annexes

Résultats jsben.ch	Algorithme 1 branche v1 boucles while, for, fonctions récursives	Algorithme 2 branche v2 méthodes de l'objet Array
recherche d'une chaîne de 3 caractères dans un tableau de 10 recettes et ventilation du résultat dans un tableau ON pour les recherches positives et OFF pour les recherches négatives <i>Méthodes <code>setSearchedRecipes()</code> + <code>getQuery()</code></i>	while, for,,,in, récursives 0,8%	includes(), filter(), concat() 100%
recherche d'une chaîne de 3 caractères dans un tableau de 10 recettes <i>Méthode <code>searchString()</code></i>	while, for,,,in 0,52%	includes() 100%
itération sur un tableau de 10 recettes	fonction récursive 94,15 %	forEach 100 %
ajout de 8 valeurs dans un tableau de 10 éléments <i>Méthode <code>addInArray()</code></i>	for,,,in 24,1 %	push(), splice(), every() 100 %
retrait de 10 valeurs dans un tableau de 20 éléments <i>Méthode <code>removeInArray()</code></i>	for,,,in 19,39 %	indexOf(), splice 100 %
intersection de 2 tableaux de 10 valeurs chacun : résultat un tableau de 5 éléments <i>Méthode <code>junctionArray()</code></i>	récursives 100 %	includes(), filter() 80,08 %
soustraction de 2 tableaux de 10 éléments chacun : résultat un tableau de 10 éléments <i>Méthode <code>subtractArray()</code></i>	récursives 100 %	includes(), filter(), concat() 96,85 %

Figure 1 – résultats du test jsben.ch

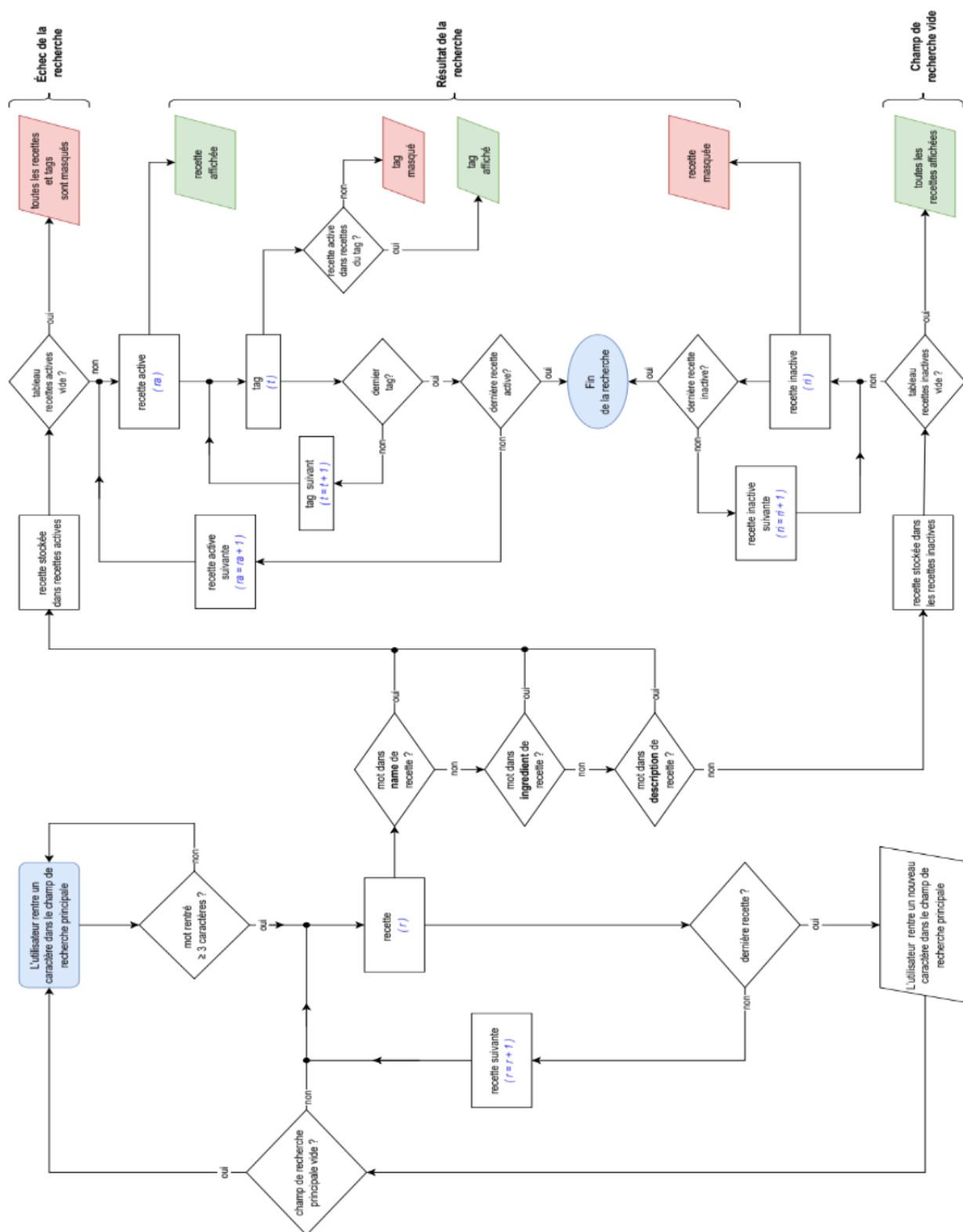


Figure 2 : algorithme du moteur de recherche